# Investigating Techniques of Monte Carlo Integration

Charles Giulian

## I. Background

Monte Carlo integration is a numerical technique for solving definite integrals using random numbers. While other numerical techniques (like the trapezoidal rule or Riemann sums) can be used to solve integrals deterministically, Monte Carlo integration is unique because it is non-deterministic. Although this means that successive runs of the same Monte Carlo algorithm will produce different results, it makes estimation of higher-order ($d \geq 4$) integrals much faster than quadrature methods. The general principle behind Monte Carlo integration is to "square off" the volume of interest, and then sample many points within that volume to approximate the integral. Differences in algorithms arise from how those random points are sampled.

Given some multidimensional definite integral:

$$I = \int_S f(\vec{x}) d\vec{x}$$

where $S$, a subset of $\mathbb{R}^m$ has volume:

$$V = \int_S d\vec{x}$$

$I$ has the approximation:

$$I \approx A = V \frac{1}{N} \sum_{i=1}^{N} f(\vec{x_i})$$

And by the law of large numbers:

$$\lim_{N \to \infty} E = I$$

## II. Uniform Monte Carlo Integration

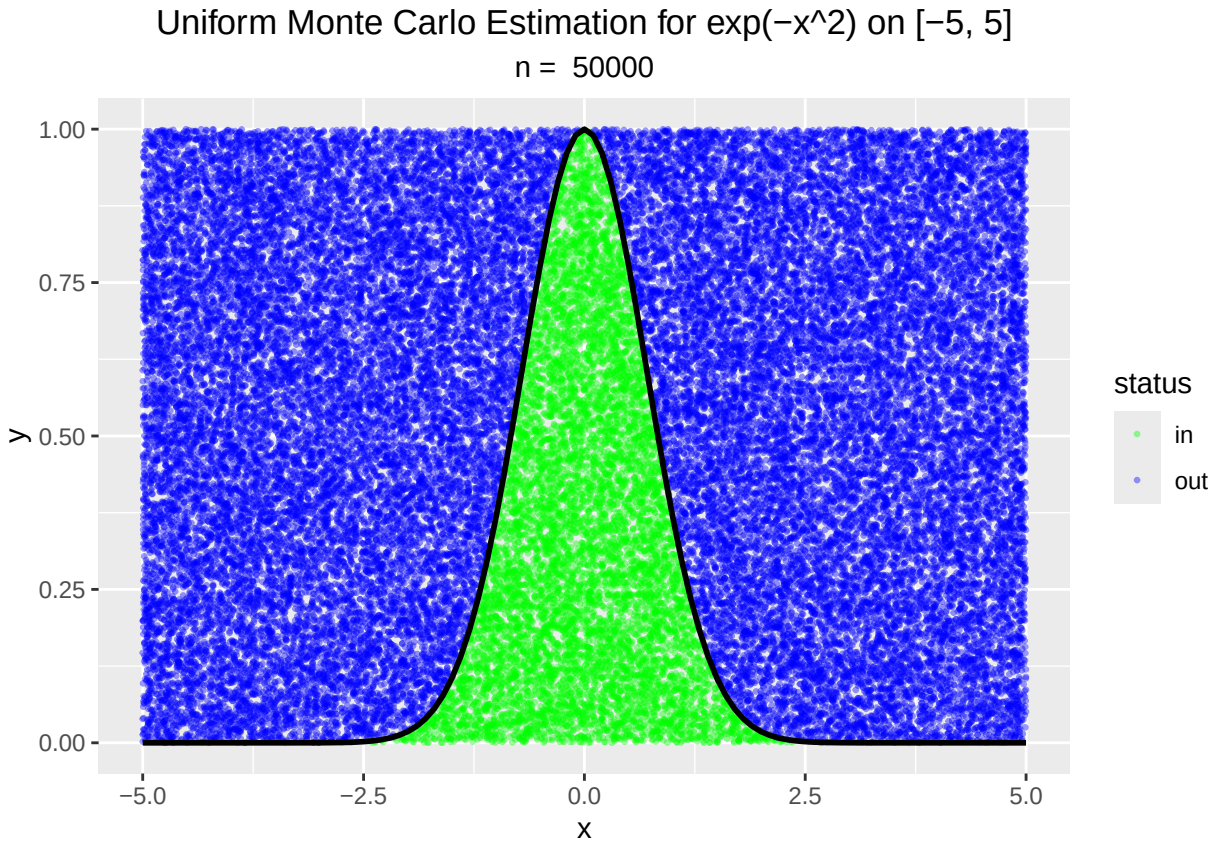
Uniform Monte Carlo integration of a one-dimensional function uniformly samples points between $a, b$ and $max(f(x))$. The following code evaluates

$$\int_{-5}^{5} e^{-x^2} dx$$

and then creates a graph to visualize points inside the region of integration (green) and those outside the region of integration (blue).

```
# Setup packages
library(ggplot2)
theme_update(plot.title = element_text(hjust = 0.5))
theme_update(plot.subtitle = element_text(hjust = 0.5))
```

```r
f <- function(x) exp(-x^2)
a <- -5
b <- 5
N <- 50000
x_vals <- seq(a, b, length.out = 1000)
y_max <- f(0)

df <- data.frame(
  x = runif(N, min = a, max = b),
  y = runif(N, min = 0, max = y_max)
)

df$status <- ifelse(df$y < f(df$x), "in", "out")

V <- (b - a) * y_max
I <- (sum(df$status == "in") / N) * V

ggplot(df, aes(x, y)) +
  geom_point(aes(color = status), size = 0.5, alpha = 0.4) +
  stat_function(fun = f, color = "black", linewidth = 1) +
  scale_color_manual(values = c("in" = "green", "out" = "blue")) +
  labs(title = "Uniform Monte Carlo Estimation for exp(-x^2) on [-5, 5]", subtitle = paste("n = ", N))
```




```r
print(paste("I = ", I))
```

```
## [1] "I =  1.7844"
```

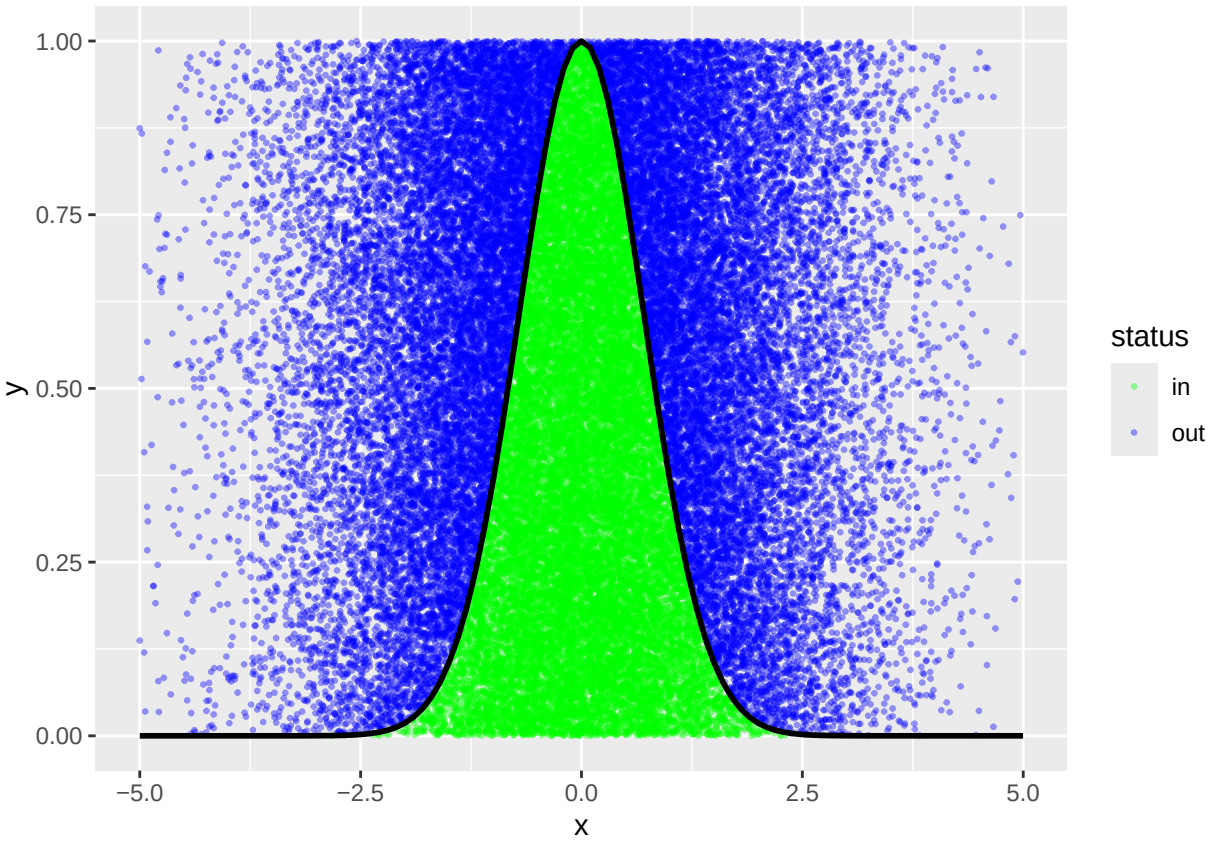
```r
print(paste("Error = ", 1.7724538509 - I))
```

```
## [1] "Error =  -0.0119461490999997"
```

This error value shows that our estimation is pretty close.

## III. Importance Sampling

```r
f <- function(x) exp(-x^2)
a <- -5
b <- 5
N <- 50000

# g(x) ~ N(0, sd=1.5)
sigma <- 1.5
x_samples <- rnorm(N, mean = 0, sd = sigma)

x_samples <- x_samples[x_samples >= a & x_samples <= b]
n_eff <- length(x_samples)

# g_pdf must be the truncated PDF (normalized to integrate to 1 over [-5, 5])
# The adjustment factor is the area of the Normal curve between -5 and 5
adjustment <- pnorm(b, 0, sigma) - pnorm(a, 0, sigma)
g_pdf <- function(x) dnorm(x, 0, sigma) / adjustment

df <- data.frame(
  x = x_samples,
  y = runif(n_eff, 0, 1)
)
df$status <- ifelse(df$y < f(df$x), "in", "out")

weights <- f(df$x) / g_pdf(df$x)
integral_estimate <- mean(weights)

ggplot(df, aes(x, y)) +
  geom_point(aes(color = status), size = 0.5, alpha = 0.4) +
  stat_function(fun = f, color = "black", linewidth = 1) +
  scale_color_manual(values = c("in" = "green", "out" = "blue"))
```




## IV. References

Press, William H., et al. Numerical Recipes: The Art of Scientific Computing. 3 ed., Cambridge University Press, 2007.

Lepage, G. Peter. "A New Algorithm for Adaptive Multidimensional Integration." Journal of Computational Physics, vol. 27, no. 2, 1978, pp. 192-203. ScienceDirect, https://doi.org/10.1016/0021-9991(78)90004-9.